

# Foreign Data Wrapper: Supporting Semijoins in PostgreSQL

---

## Team Members

---

- **Arif Ali** (23m0822)
- **Soumik Dutta** (23m0826)

Repository: [https://github.com/sammagnet7/cs631\\_DB\\_project\\_FDW\\_Semijoin.git](https://github.com/sammagnet7/cs631_DB_project_FDW_Semijoin.git)

---

## Problem Statement

---

PostgreSQL allows a relation (e.g., **takes**) on a remote site B to be defined as a foreign relation on site A. When performing **SELECT** queries on such foreign relations, PostgreSQL pushes the queries to the remote site.

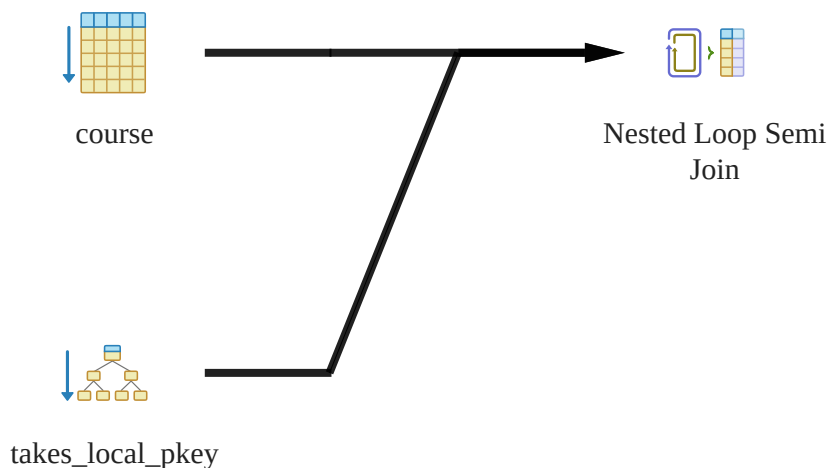
For correlated subqueries with projections on the **left** relation only, PostgreSQL performs a semijoin when both relations (**r1** and **r2**) are local.

Example query:

```
SELECT COUNT(*)  
FROM course c  
WHERE EXISTS (SELECT * FROM takes t WHERE t.year = c.year);
```

## Local Semijoin

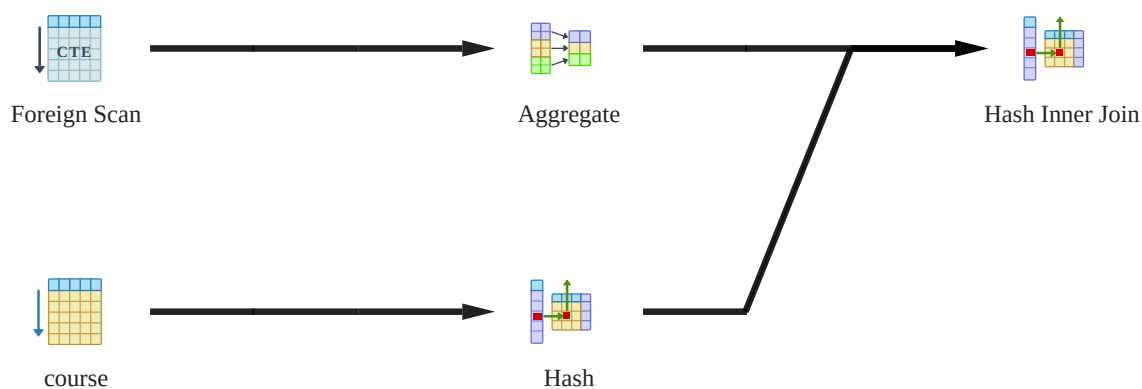
The **EXPLAIN** output for this query, when both relations are local, shows an optimized semijoin operation:



## Foreign Relation Case

However, when **r2** is on a foreign site, PostgreSQL retrieves the entire **r2** table to the local site and performs the join locally.

The **EXPLAIN** output illustrates this inefficiency:



This approach is suboptimal when the join condition matches only a small subset of rows. For example, in the query above:

- **course** has 200 rows.
- **takes** has 30,000 rows.
- Only approx 300 rows from **takes** satisfy the condition **t.year = c.year**.

Yet, all 30,000 rows are fetched from the foreign site, unnecessarily increasing network traffic.

## Solution Approach

---

The goal is to **push the semijoin to the foreign site** and fetch only the required tuples, thereby reducing network traffic.

### Semijoin Definition

1. **Semijoin Operation ( $r_1 \bowtie r_2$ ):** Selects tuples from  $r_1$  that contribute to the join  $r_1 \bowtie r_2$ .
  - **Natural Semijoin:**  $r_1 \bowtie r_2 \approx \Pi_{R_1} (r_1 \bowtie r_2)$
  - **Theta Semijoin:**  $r_1 \bowtie_{\theta} r_2 \approx \Pi_{R_1} (r_1 \bowtie_{\theta} r_2)$

### Strategy

For a semijoin ( $r_1 \bowtie r_2$ ), with join attributes  $J_1$  (from  $r_1$ ) and  $J_2$  (from  $r_2$ ):

1. Compute  $temp1 \leftarrow \Pi_{J_1} (r_1)$  at  $S_1$  (local DB).
  2. Ship  $temp1$  to  $S_2$  (foreign DB).
  3. Compute  $temp2 \leftarrow r_2 \bowtie_{(J_2=J_1)} temp1$  at  $S_2$ .
  4. Ship  $temp2$  back to  $S_1$ .
  5. Compute  $r_1 \bowtie_{(J_1=J_2)} temp2$  at  $S_1$ .
- This results in  $r_1 \bowtie r_2 \approx \Pi_{R_1} (r_1 \bowtie r_2)$ .

### Optimization with Bloom Filters

To minimize the overhead of shipping  $temp1$ :

1. Use a **Bloom filter** to encode  $J_1$  values from  $r_1$ .
2. Ship the Bloom filter to  $S_2$ .
3. At  $S_2$ , filter  $r_2$  tuples using the Bloom filter before computing  $temp2$ .

## Implementation Steps

---

### Version 1: Using **WHERE IN** Clause

- **subquery\_planner Modifications:**
  - If jointype is **JOIN\_SEMI** and is foreign scan is being performed then, a subtree path is created as a child to the existing foreignscan path.
  - This subtree path consists of a sequential scan on the local relation (the left hand relation  $r_1$  of semijoin), then we extract the unique values of the join attributes( $J_1$ ) from the scan by adding an **T\_Unique** type path node.
  - In foreign data wrapper file also we need to add this newly created path as a sub path of foreignscan path.
  - Files modified: **allpaths.c**, **postgres\_fdw.c**
- **createPlan Modifications:**

- Set proper relation scan id to cater the additionally added local sequential scan.

- **Executor Modifications:**

- As by default foreignscan can not have a subtree, we modified execForeignScan execution path to check for subtree and call execProcNode on the subtree if any.
- Adding accumulator to accumulate all the locally scanned tuples returned from `T_Unique` execution path.
- Edit and Modify the foreignScan **SELECT** query to append **WHERE** clause with **IN** parameter list. And the list is obtained by the above accumulator.
- Files modified: `nodeForeignscan.c`

**Modified Files:**

- `{PGSRC}/contrib/postgres_fdw/postgres_fdw.c`
- `{PGSRC}/src/backend/executor/execMain.c`
- `{PGSRC}/src/backend/executor/nodeForeignscan.c`
- `{PGSRC}/src/backend/optimizer/path/allpaths.c`
- `{PGSRC}/src/backend/optimizer/plan/createplan.c`
- `{PGSRC}/src/backend/optimizer/plan/planmain.c`
- `{PGSRC}/src/backend/optimizer/plan/planner.c`
- `{PGSRC}/src/backend/optimizer/util/relnode.c`
- `{PGSRC}/src/include/optimizer/optimizer.h`
- `{PGSRC}/src/include/postgres.h`
- `{PGSRC}/src/include/nodes/execnodes.h`

**Code folder:** `Codebase/semijoin_version1_inClause`

---

## Version 2: Using Bloom Filters

- **Executor Modifications:**

- Instead of sending unique values of join attributes as **WHERE IN** clause make a Bloom filter out of those, encode to hex stream and send appending it to the foreignScan query.
- Files modified: `nodeForeignscan.c`

- **Foreign Site Modifications:**

- While receiving the query including bloom filter hex string, decode stream to bloom filter and before returning the tuples, perform filtration based upon the bloom filter and finally send less number of filtered tuples from foreignDb to localDB.
- Files modified:
  - `postgres.c`, `bloom.c`, `execMain.c`

**Modified Files:**

- `{PGSRC}/src/backend/utils/misc/bloom.c`
- `{PGSRC}/src/backend/executor/execMain.c`
- `{PGSRC}/src/include/postgres.h`

- {PGRES\_SRCDIR}/src/backend/executor/nodeForeignscan.c
- {PGRES\_SRCDIR}/src/backend/tcop/postgres.c
- {PGRES\_SRCDIR}/src/backend/tcop/pquery.c
- {PGRES\_SRCDIR}/src/include/tcop/dest.h
- {PGRES\_SRCDIR}/src/include/utils/portals.h
- {PGRES\_SRCDIR}/src/backend/utils/misc/Makefile

**Code folder:** Codebase/semijoin\_version2\_bloomFilter

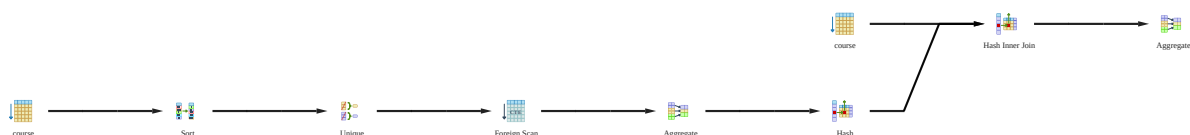
Note:

PGRES\_SRCDIR="/home/soumik/SOUMIKD/IITB/Class\_notes/sem\_3/CS\_631\_RDBMS/Assignments/Final\_project/postgres\_codebase/postgresql"

## Results

### Final EXPLAIN Output

The modified approach significantly reduces network traffic by fetching only relevant tuples from the foreign site:



## Future Work

1. Extend support for semijoins with multiple join attributes.
2. Handle multi-level recursive semijoins.
3. Dynamically adjust Bloom filter size based on join attribute estimates.
4. Improve Bloom filter transmission using UDFs.
5. Reuse PostgreSQL's existing Bloom filter implementation.

## Challenges and Shortcuts

### Challenges

- Handling foreign scans with subtrees required extensive changes in both planner and executor logic.

### Shortcuts

- Hardcoded Bloom filter parameters (e.g., size, false-positive rate).
- Limited support for multiple join attributes and recursive semijoins.

## **Desired Enhancements**

- Generalized support for semijoins with complex conditions.
- Efficient integration with PostgreSQL's existing utilities and optimizations.